CLOUD-MENU

Technical Architecture & API Specification

Phase 3: System design, data model summary, REST and real-time contracts

Version: 0.1 (draft for stakeholder review)

Date: 21 October 2026

Market: Dar es Salaam, Tanzania (launch zones: Masaki, Sinza)

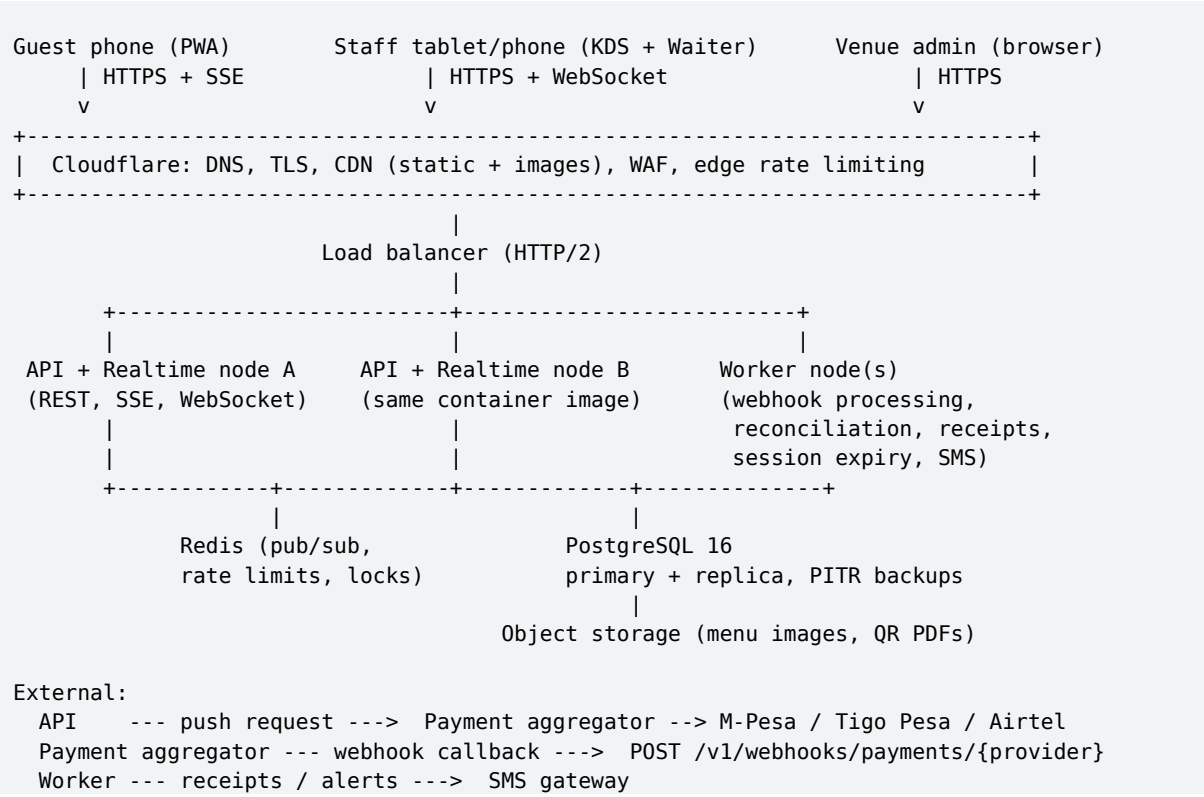
Status: Pre-development. Figures marked “target” or “hypothesis” are planning assumptions to be validated, not forecasts.

1. Design Principles

- **Asset-light:** no venue hardware; everything runs in browsers on existing phones and tablets.
- **Low-bandwidth first:** small bundles, cached app shell, compressed images, server-driven pricing so clients stay thin.
- **Money correctness over speed:** idempotent writes, single-transaction state changes, webhook-plus-polling reconciliation.
- **Boring, cheap, reliable:** a modular monolith on PostgreSQL with Redis pub/sub; split into services only when load demands it.
- **Real-time where it matters:** orders, ticket states, bill and payment updates are pushed; everything else is plain REST.

2. System Architecture

2.1 Architecture diagram



2.2 Components

Component	Responsibility
Guest PWA	QR landing, menu, cart, order tracking, bill split, payment and receipt. Service worker for shell and menu caching. SSE for live updates.
Staff app	KDS (tickets, states), waiter dashboard (tables, payments, alerts), PIN login. WebSocket with polling fallback.
Venue admin	Menu, modifiers, tables, QR sticker PDF, reports, subscription.
API + Realtime nodes	Stateless REST, SSE and WebSocket endpoints; publish and subscribe via Redis so any node can serve any client.

Component	Responsibility
Workers	Reliable background jobs: process stored webhooks, poll provider status for stale payments, expire split locks and idle sessions, send SMS and receipts.
PostgreSQL	System of record, including the realtime_events log used for resync after disconnects.
Redis	Pub/sub fan-out, rate-limit counters, short-lived locks. Loss of Redis must not lose data (state is in PostgreSQL).
Payment aggregator	Single integration for M-Pesa, Tigo Pesa and Airtel Money push prompts and callbacks (vendor selected in Sprint 0).

2.3 Technology stack recommendations

Layer	Recommendation	Rationale	Alternative
Guest front-end	Preact + Vite + TypeScript, Tailwind (purged), Workbox service worker	Very small runtime keeps initial JS under 150 KB gzipped; strong PWA tooling	SvelteKit
Staff / admin front-end	Same stack and shared component library (Preact or React)	One design system, shared types, fewer skills needed	React + TanStack Query
Backend	Node.js (current LTS) + TypeScript + Fastify, modular monolith	High I/O concurrency, native SSE/WebSocket support, one language across the stack	NestJS, Go
Real-time	SSE for guests; WebSocket for staff; Redis pub/sub between nodes	SSE is one-way, proxy-friendly and auto-reconnects (ideal for guests); WebSocket gives staff two-way acks	Managed realtime (Ably, Pusher) if ops capacity is thin
Database	PostgreSQL 16 (managed) behind PgBouncer	Strong transactions and constraints for money; JSONB for provider payloads	None recommended for MVP
Jobs/queue	pg-boss (queue inside PostgreSQL)	No extra infrastructure; transactional enqueue with the outbox pattern	BullMQ on Redis
Cache / pub-sub	Redis (small managed instance)	Fan-out and rate limiting	Postgres LISTEN/NOTIFY for a single-node start
Storage / CDN	Cloudflare (CDN, WAF) + object storage for images, WebP/AVIF variants	East Africa edge presence (verify), low egress cost, DDoS protection	CloudFront + S3
Hosting	Containers (Docker) on managed container service or two VPS nodes; Terraform for IaC	Cheap to start, simple to scale horizontally	Fly.io, Render
Region	Default: AWS af-south-1 (Cape Town). Measure RTT from Dar es Salaam networks (Vodacom, Airtel, Yas) to candidate regions in Sprint 0	Closest major cloud region; EU low-cost providers are an alternative if latency is acceptable	Hetzner/OVH EU
Observability	OpenTelemetry, Sentry, Grafana (or managed equivalent), uptime probes	Trace order and payment flows end to end	Datadog (costlier)
CI/CD	GitHub Actions: lint, tests, Lighthouse CI budget, container build, staged deploy	Performance budget enforced on every PR	GitLab CI
SMS	Local-capable SMS gateway (for receipts, alerts)	Fallback when guest closes the browser	WhatsApp Business API (post-MVP)

3. Real-Time Synchronisation Design

Concern	Design
Guest channel	SSE: GET /v1/sessions/{id}/events. Server sends events with id: <seq>; browser reconnects automatically with Last-Event-ID so missed events are replayed from realtime_events.
Staff channel	WebSocket: wss://{api}/v1/staff/ws. Client sends hello with last_seq; server replays missed events, then streams live. Client acks critical events (new ticket).
Fan-out	Writers insert into realtime_events inside the business transaction (outbox), then publish seq to Redis channel venue:{id}. Every node pushes to its connected clients.
Ordering and dedupe	seq is monotonic per database; clients ignore events with seq lower than the last applied.
Weak networks	Heartbeat every 20 s; client declares offline after 2 missed beats, shows banner, retries with exponential backoff (1 s to 15 s) and falls back to polling GET /v1/staff/orders?since_seq= every 5 s after 3 failures.
Capacity guideline	A single small Node instance comfortably holds thousands of idle SSE/WebSocket connections; load-test target for MVP is 300 venues x 30 live connections (9,000) with p95 event delivery under 2 s.

4. Core Flows and State Machines

4.1 Payment flow (push prompt and webhook)

- Client calls POST /v1/shares/{id}/payments with an Idempotency-Key. Server validates split is LOCKED and the share is UNPAID.
- Server inserts payment_transactions (INITIATED) in one transaction, then calls the aggregator (timeout 10 s). On accepted response: status PENDING, store provider_reference, return 202.
- Guest enters PIN on the phone. Aggregator calls POST /v1/webhooks/payments/{provider}.
- Webhook handler: verify signature and source, store raw event (dedupe on provider + event_id), respond 200 immediately, enqueue processing.
- Worker applies the result in one database transaction: lock transaction row, set SUCCESS/FAILED, mark share PAID, update bill paid_tzs, issue receipt, write realtime events. Guest SSE and staff WebSocket receive the update.
- Reconciliation worker polls the aggregator for any transaction PENDING longer than 3 minutes (then with backoff until expiry at about 10 minutes). Late success after expiry is recorded and flagged needs_review.

Payment transaction states

From	To	Trigger
INITIATED	PENDING	Aggregator accepted request
INITIATED	FAILED	Aggregator rejected or timed out without acceptance
PENDING	SUCCESS	Verified webhook or reconciliation poll reports success
PENDING	FAILED	Provider reports failure (insufficient funds, wrong PIN, declined)
PENDING	CANCELLED	Customer cancelled on handset
PENDING	EXPIRED	No final state before expires_at (a later success is still accepted and flagged)

Order and ticket states

Order item: NEW → ACCEPTED → PREPARING → READY → SERVED; any state before SERVED → REJECTED (reason required) or CANCELLED. Order status is derived from its items. Transitions are validated server-side; invalid jumps return 409 INVALID_TRANSITION. First writer wins when two devices act simultaneously.

Split states

OPEN → LOCKED (10-minute lock; extended while any payment is in flight) → SETTLED when all shares are PAID. LOCKED → EXPIRED (no payments in flight) or CANCELLED (explicit unlock). Only one OPEN/LOCKED split per bill (enforced by a unique partial index).

4.2 Bill-split algorithm and rounding

All arithmetic uses integer shillings. Rounding rules are deterministic so every client and the server compute the same numbers.

```
EQUAL(total, n):
    base = floor(total / n)
    r    = total - base * n                    # leftover shillings
    share[i] = base + (1 if i < r else 0)      # i = 0..n-1, ordered by slot index
    # 100,000 / 3 -> 33,334 | 33,333 | 33,333   (sum = total)

ITEMIZED(bill, allocations):
    S  = sum of all item amounts (subtotal)
    E  = bill.total - S                        # service charge and any exclusive tax
    for share i:  a_i = sum of item amounts allocated to i
    extra_i = floor(E * a_i / S)               # proportional extras
    left    = E - sum(extra_i)
    give +1 shilling to the 'left' shares with the largest remainder of (E*a_i mod S),
        ties broken by share index
    share_amount_i = a_i + extra_i
    Lock allowed only if sum(a_i) == S (nothing unassigned) and sum(share_amount_i) ==
bill.total

SHARED ITEM (price P among k guests): EQUAL(P, k) with the same leftover rule.
```

4.3 Concurrency rules

- Any change to a split (allocations, lock, unlock) runs in a transaction that first takes SELECT … FOR UPDATE on the bill_splits row.
- A new order while a split is LOCKED updates the bill version but does not alter the locked split; it appears as an additional open balance and needs a new split after settlement.
- Splits store bill_version; locking fails with 409 BILL_CHANGED if the bill changed since the split was computed.

5. Infrastructure, Reliability and Cost Control

Topic	Approach
Environments	dev (local Docker), staging (mirrors prod at small size, payment sandbox), production.
MVP topology	2 API/real-time nodes + 1 worker + managed PostgreSQL (primary with automated backups and point-in-time recovery) + small Redis. Add a read replica and a second worker when reporting or job load requires.
Scaling	Scale API nodes horizontally (stateless). Use PgBouncer for connection pooling. Move reports to a replica. Partition realtime_events by month if volume grows.

Topic	Approach
Availability targets	MVP: 99.5% monthly for ordering and payments. Targets: RPO at most 5 minutes (PITR), RTO at most 60 minutes. Review after pilot.
Cost levers	Single region; small instances; image variants served from CDN; pg-boss instead of a separate queue; managed services only where they save engineering time. Price the stack in Sprint 0 using vendor quotes.
Deployments	Rolling deploys with health checks; backward-compatible migrations; feature flags for risky changes; deploy outside Friday and Saturday evening peak.
Backups and DR	Daily snapshots plus PITR; quarterly restore test; infrastructure defined in Terraform.

6. Security and Compliance

- **QR integrity:** QR URL carries qr_code plus HMAC signature (key version stored per table). Rotating the key version invalidates old stickers.
- **Guest identity:** no account; session-scoped token (short-lived JWT or opaque token) bound to session and participant. Cannot read other sessions (403).
- **Staff auth:** venue code + PIN, hashed with Argon2/bcrypt, lockout after 5 failures, role-based access (WAITER, KITCHEN, BAR, MANAGER, OWNER).
- **Payments:** idempotency keys on all money-moving endpoints; webhook signature and source IP verification; replay protection via unique provider event id; no card data handled.
- **Personal data:** phone numbers encrypted at application level, masked in logs, retained only as needed for payments and receipts. Review obligations under Tanzania's Personal Data Protection Act, 2022 with counsel.
- **Fiscal and tax:** confirm with TRA or a tax adviser whether venues must issue fiscal (EFD/VFD) receipts for these sales and how the platform's digital receipt should relate to them. This can affect scope.
- **Platform:** TLS everywhere, WAF and rate limits at the edge, dependency scanning, secrets in a secret manager, audit log for staff overrides and cash payments.

7. Observability

- Correlation id per request propagated to jobs and real-time events; structured JSON logs with PII masked.
- Business dashboards: orders per minute, order-to-ticket latency, payment success rate by provider, webhook lag, pending-payment age, SSE/WebSocket connections.
- Alerts: payment success rate drops, webhook failures, pending payments older than 10 minutes, queue depth, p95 API latency, error rate, database connections.

8. Data Model Summary

Full PostgreSQL DDL is delivered separately in **03_cloud_menu_schema.sql** (validated for syntax). Money is stored as BIGINT TZS; rates in basis points.

Domain	Tables	Key constraints
Tenancy	venues, subscriptions, staff_users	venues.slug unique; staff scoped by venue_id; PIN stored hashed
Floor	dining_tables	unique (venue_id, label); unique qr_code; qr_key_version for rotation

Domain	Tables	Key constraints
Menu	menus, categories, items, modifier_groups, modifiers, item_modifier_groups	price_tzs >= 0; modifier group min/max checks; items.is_available for sold-out
Sessions	table_sessions, session_participants	one ACTIVE session per table (partial unique index); unique (session_id, device_hash)
Orders	orders, order_items, order_item_modifiers, realtime_events	unique (session_id, client_order_id) for idempotency; name and price snapshots; line total check
Billing	bills, bill_splits, bill_shares, bill_share_items	one live split per bill; allocation guard trigger; bill version for optimistic locking
Payments	payment_transactions, payment_webhook_events, receipts	unique idempotency_key and our_reference; one in-flight and one success per share; webhook dedupe on (provider, event_id)
Platform	idempotency_keys, audit_log	composite key (scope, key); append-only audit

9. API Specification

9.1 Conventions

Topic	Convention
Base URL	https://api.{domain}/v1 (domain placeholder)
Format	JSON, UTF-8. Money as integer TZS. Times as ISO-8601 UTC. IDs are UUIDs.
Guest auth	Authorization: Bearer <session_token> returned by session init
Staff auth	Authorization: Bearer <staff_token> (role claim)
Idempotency	Idempotency-Key header (UUID) required on POST orders, splits lock and payments; same key and body returns the original response for 24 h; same key with a different body returns 422.
Errors	RFC 7807 application/problem+json with a stable machine code, e.g. {"type":"https://api.{domain}/errors/ITEM_UNAVAILABLE","title":"Item unavailable","status":409,"code":"ITEM_UNAVAILABLE","detail":"...","request_id":"..."}
Caching	GET menu supports ETag / If-None-Match and Cache-Control: max-age=60, stale-while-revalidate=300
Rate limits	Session init 30/min/IP; order submit 10/min/session; payment initiate 5/min/share; 429 with Retry-After
Versioning	URL major version; additive changes only within v1

9.2 Endpoint overview

Method and path	Auth	Purpose
POST /v1/sessions/init	None (QR token)	Create or join a table session
GET /v1/venues/{venue_id}/menu	Guest	Menu with categories, items, modifiers (ETag)
POST /v1/sessions/{id}/orders	Guest	Submit order (idempotent)
GET /v1/sessions/{id}/bill	Guest	Current bill with line items and totals
POST /v1/sessions/{id}/splits	Guest	Create or recalculate a split (EQUAL or ITEMIZED)
PUT /v1/splits/{id}/allocations	Guest	Set itemized allocations
POST /v1/splits/{id}/lock	Guest	Validate and lock the split

Method and path	Auth	Purpose
POST /v1/splits/{id}/unlock	Guest/Staff	Cancel lock if no payment in flight
POST /v1/shares/{id}/payments	Guest	Initiate mobile money push prompt
GET /v1/payments/{id}	Guest	Payment status (fallback to SSE)
GET /v1/shares/{id}/receipt	Guest or public token	Digital receipt
POST /v1/webhooks/payments/{provider}	Signature	Provider callback
GET /v1/sessions/{id}/events	Guest	SSE stream
POST /v1/staff/auth/login	None	Venue code + PIN login
GET /v1/staff/orders?status=&station=&since_seq=	Staff	Ticket list and resync
PATCH /v1/staff/order-items/{id}	Staff	Change item state / reject with reason
PATCH /v1/staff/items/{id}/availability	Staff	Sold-out toggle
GET /v1/staff/tables	Staff	Tables with session, balance and shares paid
POST /v1/staff/shares/{id}/cash-payment	Waiter+	Record cash payment
POST /v1/staff/sessions/{id}/close	Waiter+	Close session with reason
GET /v1/ws (WebSocket)	Staff	Real-time channel

9.3 Initialise table session

POST /v1/sessions/init: creates the table session on first scan or joins the active one. Idempotent per device.

Request

```
{
  "qr": "k7Tq9xP2.Zm4dQ1wL",              // qr_code.signature from the QR URL
  "device_id": "d5b1c2e0-7a41-4f6d-8c3e-0b9a1d2f3e44",   // random id generated by the PWA
  "locale": "sw"
}
```

Response 201 (new session) / 200 (joined or re-scan)

```
{
  "session": {
    "id": "0f8e4c1a-3b2d-4c5e-9a71-2d6f8b9e1c01",
    "status": "ACTIVE",
    "opened_at": "2026-10-23T18:42:10Z"
  },
  "venue": { "id": "a31f6d7e-92b0-4a85-b3c6-5e0d21f4a8b2", "name": "Masaki Lounge",
             "currency": "TZS", "menu_version": 14 },
  "table": { "id": "c9a2e7d4-1f3b-4e68-8a05-7d4c3b2a1e90", "label": "T12" },
  "participant": { "id": "5e2b9c7a-4d1f-4a38-b6e0-9c8d7a6b5f41", "display_name": "Guest 2" },
  "session_token": "eyJhbGciOi...",
  "token_expires_at": "2026-10-24T00:42:10Z"
}
```

Errors: 404 QR_NOT_FOUND, 422 QR_SIGNATURE_INVALID, 403 VENUE_SUSPENDED, 409 SESSION_FULL, 429 RATE_LIMITED.

9.4 Submit guest order

POST /v1/sessions/{session_id}/orders (headers: Authorization, Idempotency-Key). The server re-prices every line from current menu data; client prices are ignored.

Request

```
{
```

```
  "client_order_id": "6b0f6c52-5d5b-4b77-9e0b-1f6a4d7c2e10",
  "note": "Birthday table",
  "items": [
    { "item_id": "e1a4c3b2-8d7f-4a69-b5c0-3f2e1d0c9b8a", "quantity": 2,
      "modifiers": [ { "modifier_id": "f2b5d4c3-9e80-4b7a-a6d1-4a3f2e1d0c9b" } ],
      "note": "No ice" },
    { "item_id": "d0c9b8a7-6f5e-4d3c-b2a1-0f9e8d7c6b5a", "quantity": 1, "modifiers": [] }
  ]
}
```

Response 201

```
{
  "order": {
    "id": "7c6b5a49-3827-4165-a4f3-e2d1c0b9a897",
    "order_no": 87,
    "status": "NEW",
    "created_at": "2026-10-23T18:47:31Z",
    "items": [
      { "id": "11111111-2222-4333-8444-555555555551", "name": "Vodka + Mixer", "quantity": 2,
        "unit_price_tzs": 8000, "modifiers_total_tzs": 3000, "line_total_tzs": 22000,
        "station": "BAR", "status": "NEW" },
      { "id": "11111111-2222-4333-8444-555555555552", "name": "Grilled Tilapia", "quantity":
1,
        "unit_price_tzs": 25000, "modifiers_total_tzs": 0, "line_total_tzs": 25000,
        "station": "KITCHEN", "status": "NEW" }
    ],
    "subtotal_tzs": 47000
  },
  "repriced": false
}
```

Errors: 409 ITEM_UNAVAILABLE (with item ids), 422 MODIFIER_RULE_VIOLATION, 409 SESSION_CLOSED, 422 VALIDATION_ERROR. Replay with the same Idempotency-Key returns 200 with the original order.

9.5 Calculate, allocate and lock a split

POST /v1/sessions/{session_id}/splits creates (or replaces the OPEN) split and returns computed shares.

Request: equal split

```
{ "mode": "EQUAL", "payer_count": 3 }
```

Response 201

```
{
  "split": {
    "id": "9d8c7b6a-5f4e-4d3c-a2b1-c0d9e8f7a6b5",
    "mode": "EQUAL", "status": "OPEN",
    "bill_version": 5, "total_tzs": 100000, "payer_count": 3,
    "shares": [
      { "id": "aa000001-0000-4000-8000-000000000001", "label": "Guest 1", "amount_tzs":
33334, "status": "UNPAID" },
      { "id": "aa000001-0000-4000-8000-000000000002", "label": "Guest 2", "amount_tzs":
33333, "status": "UNPAID" },
      { "id": "aa000001-0000-4000-8000-000000000003", "label": "Guest 3", "amount_tzs":
33333, "status": "UNPAID" }
    ],
    "unassigned_tzs": 0
  }
}
```

Request: itemized split (PUT /v1/splits/{id}/allocations)

```
{
  "mode": "ITEMIZED",
  "shares": [
```

```
    { "share_id": "aa000002-0000-4000-8000-000000000001", "participant_id": "5e2b9c7a-4d1f-
4a38-b6e0-9c8d7a6b5f41",
      "items": [ { "order_item_id": "11111111-2222-4333-8444-555555555552", "amount_tzs":
25000 } ] },
    { "share_id": "aa000002-0000-4000-8000-000000000002", "participant_id": "6f3c0d8b-5e20-
4b49-87f1-0d9e8b7c6a52",
      "items": [ { "order_item_id": "11111111-2222-4333-8444-555555555551", "amount_tzs":
11000 } ] }
  ]
}
```

Response returns each share with amount_tzs (items plus proportional service charge and tax) and unassigned_tzs. Error 409 ALREADY_CLAIMED if an allocation exceeds the line total.

Lock: POST /v1/splits/{id}/lock

```
// Request
{ "bill_version": 5 }

// Response 200
{
  "split": { "id": "9d8c7b6a-5f4e-4d3c-a2b1-c0d9e8f7a6b5", "status": "LOCKED",
             "locked_at": "2026-10-23T19:10:02Z", "lock_expires_at": "2026-10-23T19:20:02Z" }
}
```

Errors: 409 BILL_CHANGED, 409 SPLIT_ALREADY_LOCKED, 422 UNASSIGNED_AMOUNT (itemized with leftovers), 409 SESSION_CLOSED.

9.6 Initiate mobile money payment (push prompt)

POST /v1/shares/{share_id}/payments (headers: Authorization, Idempotency-Key)

Request

```
{
  "provider": "MPESA",                    // MPESA | TIGOPESA | AIRTELMONEY
  "msisdn": "255754123456",               // E.164 without +
  "tip_tzs": 0
}
```

Response 202 Accepted

```
{
  "payment": {
    "id": "bb000003-0000-4000-8000-000000000001",
    "share_id": "aa000001-0000-4000-8000-000000000001",
    "provider": "MPESA",
    "amount_tzs": 33334,
    "status": "PENDING",
    "expires_at": "2026-10-23T19:22:30Z",
    "instructions": "Check your phone and enter your PIN to approve TZS 33,334."
  }
}
```

Errors: 422 INVALID_MSISDN_FOR_PROVIDER, 409 SPLIT_NOT_LOCKED, 409 SHARE_ALREADY_PAID, 409 PAYMENT_IN_FLIGHT (returns the existing payment), 502 PROVIDER_UNAVAILABLE (transaction marked FAILED, safe to retry), 429 RATE_LIMITED.

9.7 Payment provider callback webhook

POST /v1/webhooks/payments/{provider}. The payload below is illustrative; map it to the chosen aggregator's real format and signature scheme during integration.

```
// Headers
X-Signature: t=1761245402,v1=3f9a...c21        // HMAC-SHA256 over "t.body" with the shared
secret
X-Event-Id: evt_01JBXYZ...
```

```
// Body
{
  "event_id": "evt_01JBXYZ...",
  "type": "payment.completed",                 // payment.completed | payment.failed |
payment.cancelled
  "reference": "CM-20261023-8F3K2A",           // our_reference sent at initiation
  "provider_reference": "QJK7XXXX01",
  "amount": 33334,
  "currency": "TZS",
  "status": "SUCCESS",
  "failure_code": null,
  "timestamp": "2026-10-23T19:12:44Z"
}

// Response (always fast, after persisting the raw event)
200 OK    {"received": true}
401       invalid signature or source (logged as security event)
```

- Processing rules: verify signature and timestamp tolerance (5 minutes); dedupe on (provider, event_id); match our_reference and amount; ignore if already in a final state; apply transition in one transaction.
- Respond 200 for duplicates and unknown-but-valid references (to stop retries) while logging them; respond 5xx only if the event could not be stored.

9.8 Staff endpoints (selected payloads)

PATCH /v1/staff/order-items/{id}

```
// Request
{ "status": "READY" }
// or
{ "status": "REJECTED", "reason": "Out of stock" }

// Response 200
{ "id": "11111111-2222-4333-8444-555555555552", "status": "READY", "order_status": "READY",
"seq": 48213 }

// 409 INVALID_TRANSITION
{ "code": "INVALID_TRANSITION", "current_status": "SERVED" }
```

GET /v1/staff/tables

```
{
  "tables": [
    { "table_id": "c9a2e7d4-1f3b-4e68-8a05-7d4c3b2a1e90", "label": "T12",
      "session_id": "0f8e4c1a-3b2d-4c5e-9a71-2d6f8b9e1c01",
      "bill_total_tzs": 100000, "paid_tzs": 66667, "balance_tzs": 33333,
      "shares": [ { "label": "Guest 1", "status": "PAID" }, { "label": "Guest 2", "status":
"PAID" },
                  { "label": "Guest 3", "status": "PENDING" } ],
      "ready_items": 1, "waiter_called": false }
  ],
  "seq": 48213
}
```

10. Real-Time Event Specification

10.1 Guest SSE: GET /v1/sessions/{id}/events

```
id: 48213
event: order.status_changed
data: {"order_id":"7c6b5a49-...","item_id":"11111111-...","status":"PREPARING"}

id: 48220
```

```
event: payment.updated
data:
{"payment_id":"bb000003-...","share_id":"aa000001-...","status":"SUCCESS","paid_at":"2026-10-23T19:12:45Z"}
```

Event	Payload essentials	Trigger
menu.item_availability	item_id, is_available	Staff toggles sold-out
order.created	order_id, order_no, items	Any participant submits an order
order.status_changed	order_id, item_id, status, reason?	KDS changes state
bill.updated	bill_version, total_tzs, balance_tzs	New order or rejected item
split.updated / split.locked / split.unlocked	split_id, shares[], lock_expires_at	Split edited or locked
payment.updated	payment_id, share_id, status, failure_code?	Webhook or reconciliation result
bill.settled	bill_id	All shares paid
session.closed	reason	Auto close or waiter close

10.2 Staff WebSocket: wss://api.{domain}/v1/ws

```
// Client -> Server (after connecting with ?token=<staff_token>)
{ "type": "hello", "venue_id": "a31f6d7e-...", "station": "KITCHEN", "last_seq": 48200 }
{ "type": "ack", "seq": 48213 }                              // critical events (new ticket)
{ "type": "order_item.update", "id": "11111111-...", "status": "ACCEPTED", "req_id": "r-91" }
{ "type": "ping" }

// Server -> Client
{ "type": "hello.ok", "server_time": "2026-10-23T19:00:00Z", "replaying": 3 }
{ "type": "ticket.created", "seq": 48213, "order": { "id": "7c6b5a49-...", "order_no": 87,
    "table": "T12", "items": [ { "id": "1111...", "name": "Grilled Tilapia", "qty": 1,
"note": "No onions" } ] } }
{ "type": "ticket.updated", "seq": 48214, "item_id": "1111...", "status": "PREPARING" }
{ "type": "payment.updated", "seq": 48220, "table": "T12", "share": "Guest 3", "status":
"SUCCESS" }
{ "type": "waiter.called", "seq": 48225, "table": "T12" }
{ "type": "result", "req_id": "r-91", "ok": true }
{ "type": "error", "req_id": "r-92", "code": "INVALID_TRANSITION" }
{ "type": "pong" }
```

- Close codes: 4401 token expired (re-login), 4403 role not permitted, 4429 too many connections per user.
- Resync: if last_seq is older than the retention window (30 days) or missing, the server sends snapshot.required and the client reloads via REST.

11. Open Decisions for Sprint 0

#	Decision	Owner	Needed by
1	Payment aggregator selection (coverage of all three wallets, push support, callbacks, settlement time, fees, sandbox quality, support SLA)	Founder + Tech Lead	Week 0
2	Cloud region and latency test from Dar es Salaam networks	Tech Lead	Week 1
3	Tax and fiscal receipt treatment; personal data registration requirements	Founder + adviser	Before beta
4	Service charge and VAT display rules per venue (inclusive vs exclusive)	Product Owner	Sprint 2
5	Settlement model: aggregator pays venues directly vs platform collects and disburses (affects licensing and risk)	Founder + adviser	Week 1
6	Swahili and English copy approach and translation owner	Product Owner	Sprint 1